분산 키-값 저장소 구현 요약

개요

이 문서는 Rust **libp2p**와 **Kademlia DHT**를 활용하여 구현한 분산 키-값 저장소(distributed key-value store)의 내용을 요약합니다. 이 구현은 피어 간 네트워크에서 키-값 쌍을 저장하고 검색하기 위해 DHT를 사용하는 실제적인 P2P 애플리케이션의 동작 방식을 보여줍니다.

구현 상태

✅ **완료됨**: 전체 구현 및 상세 문서화
⚠️ **참고**: Windows 환경에서는 Visual Studio Build Tools 필요

구현된 주요 기능

1. 핵심 아키텍처

```
#[derive(NetworkBehaviour)] struct KvStoreBehaviour { kademlia: kad::Behaviour<MemoryStore>, identify: identify::Behaviour, }
```

구성 요소:

- **Kademlia DHT**: 분산 해시 테이블 기반 키-값 저장
- **Identify 프로토콜**: 피어 탐색 및 정보 교환
- **Noise 보안**: 피어 간 암호화 통신
- **Yamux 멀티플렉싱**: 단일 연결에서 다중 스트림

- **TCP 전송**: 신뢰성 있는 네트워크 통신

---

2. 명령 인터페이스

CLI 명령어:

- `put <key> <value>` - 키-값 쌍 저장
- `get <key>` - 키로 값 검색
- `list` - 저장된 레코드 조회
- `connect <addr>` - 특정 피어에 연결
- `quit` - 프로그램 종료

---

3. DHT 동작

PUT 연산:

```
    key: kad::RecordKey::new(&key),
    value: value.into_bytes(),
    publisher: None,
    expires: None,
};

swarm.behaviour_mut().kademlia.put_record(record, kad::Quorum::One)
```

GET 연산:

```
let query_id =
swarm.behaviour_mut().kademlia.get_record(kad::RecordKey::new(&key));
```

---

4. 이벤트 처리 시스템

Swarm 이벤트:

- 새로운 리스닝 주소
- 연결 성립 및 종료
- 프로토콜별 이벤트

Kademlia 이벤트:

- 쿼리 진행 상황 및 결과
- 라우팅 테이블 갱신
- 인바운드 요청 처리

Identify 이벤트:

- 피어 정보 교환
- 주소 탐색

기술적 구현 세부사항

네트워크 스택

```
애플리케이션 계층 (CLI 명령어)
        ↓
Swarm 계층 (이벤트 루프 & 연결 관리)
        ↓
네트워크 동작 (Kademlia + Identify)
        ↓
프로토콜 스택 (Noise + Yamux + TCP)
```

키 분산 원리

Kademlia는 XOR 거리 기반으로 키를 분산:

1. **키 해싱**: 키를 256비트 식별자로 해싱

2. **거리 계산**: 키 해시와 노드 ID 간 XOR 거리 계산

3. **가까운 노드 선택**: k개 가까운 노드에 레코드 저장

4. **복제**: 여러 노드에 레코드 복제 저장

쿼리 처리 과정

GET/PUT 모두 **반복적(Iterative) 쿼리** 사용:

1. **부트스트랩**: 가장 가까운 피어 집합으로 시작

2. **병렬 쿼리**: 여러 피어에 동시 질의

3. **점진적 갱신**: 더 가까운 피어를 찾아 이동

4. **종료**: 더 가까운 피어가 없을 때 종료

라우팅 테이블 관리

- **k-buckets**: 거리 범위별 피어 그룹
- **버킷 크기**: 최대 k개 피어 저장 (일반적으로 20)
- **교체 정책**: LRU 기반 교체
- **유지 관리**: 주기적 ping으로 피어 생존 확인

코드 구조

- `Cargo.toml` - 프로젝트 설정
- `src/main.rs` - 메인 구현 (300+ 라인)
- `README.md` - 사용자 문서

- `IMPLEMENTATION_GUIDE.md` - 기술 구현 가이드
- `run_demo.sh` - 데모 실행 스크립트

사용 예시

노드 실행

터미널 1:

```
cargo run -- --port 4001
```

터미널 2:

```
cargo run -- --port 4002 --connect /ip4/127.0.0.1/tcp/4001
```

세션 예시

`put hello world
✓ 저장됨: hello

get hello
✓ 조회됨: hello = world`

학습 성과

1. **NetworkBehaviour 조합**
    - Kademlia + Identify 결합
    - 이벤트 기반 프로토콜 상호작용
2. **Kademlia DHT 활용**
    - 분산 키-값 저장소 설계

    - 반복적 쿼리 알고리즘 이해

3. **전송 스택 구성**
    - Noise 보안, Yamux 멀티플렉싱
    - 연결 관리 방식
4. **비동기 이벤트 처리**
    - 이벤트 기반 아키텍처
    - 병렬 쿼리 처리
5. **P2P 네트워크 동역학**
    - 피어 탐색과 네트워크 회복력
    - 자율적 네트워크 구성

성능 특성

- **확장성**: O(log N) 복잡도
- **메모리 사용**: 메모리 기반 저장소 + 제한된 라우팅 테이블
- **네트워크 효율**: 연결 재사용 + 경량 메시지

보안 고려사항

- **전송 보안**: Noise 프로토콜 기반 암호화
- **피어 인증**: 암호학적 ID
- **한계점**:
    - 접근 제어 없음

    - 값 암호화 없음
    - Sybil 공격 취약

확장 가능 기능

- **지속성 저장소** (디스크 기반)
- **인증된 레코드** (서명 포함)
- **값 암호화** (대칭키 암호화)
- **웹 인터페이스** (REST, WebSocket)
- **고급 기능** (TTL, 접근 제어, 모니터링)

다른 구현체와 비교

- **Redis Cluster**: 중앙 집중 없음, 자동 피어 탐색
- **Cassandra**: 가벼움, P2P 네이티브 구조
- **IPFS**: 단순화된 키-값 저장소에 더 적합

결론

이 구현은 다음을 입증합니다:

1. **실용적 P2P 애플리케이션**: libp2p 활용
2. **DHT 이해**: Kademlia의 작동 원리
3. **프로토콜 조합**: 다중 프로토콜 통합

4. **비동기 이벤트 아키텍처**: 네트워크 이벤트 처리

5. **분산 시스템 기초 학습**: 확장성과 회복력 이해